1. Singleton Class

Introduction

- Singleton is a design pattern used in Java.
- It ensures that only one instance of a class is created.
- It provides a common access point to that instance.
- It is useful when a single shared object is required.
- It is commonly used in logging and configuration management.

Main features:

- Private constructor prevents external object creation.
- Static variable stores the single object.
- Static method returns the object.

Example:

```
class Singleton {
    private static Singleton obj = new Singleton();

    private Singleton() {}

    public static Singleton getInstance() {
        return obj;
    }
}

class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance();

        System.out.println(s1 == s2);
    }
}
```

Output: `true`

2. Java Reflection API

Introduction

- Reflection is a feature of Java used at runtime.
- It allows a program to examine its own classes.
- It can access information about methods and fields.
- It can inspect constructors and class names.
- It is widely used in frameworks and testing tools.

Example:

```
class Student {
    public void display() {}
}

class Main {
    public static void main(String[] args) {
        Class<?> c = Student.class;
        System.out.println(c.getName());
    }
}
```

Output: `Student`

Important methods:

- `getName()` – gets the class name.
- `getDeclaredMethods()` – gets declared methods.
- `getDeclaredFields()` – gets declared fields.
- `getDeclaredConstructors()` – gets declared constructors.

3. Lambda Expressions

Introduction

- Lambda expressions were introduced in Java 8.
- They provide a shorter way to write implementations.
- They are used with functional interfaces.
- They reduce unnecessary code.
- They are useful in collections and event handling.

Syntax:

```
(parameters) -> expression
```

Example:

```
interface Addition {
    int add(int a, int b);
}

class Main {
    public static void main(String[] args) {
        Addition obj = (a, b) -> a + b;
        System.out.println(obj.add(10, 20));
    }
}
```

Output: `30`

4. Functional Interfaces

Introduction

- A functional interface contains exactly one abstract method.
- It can have multiple `default` and `static` methods.
- It can be implemented using a lambda expression.
- The `@FunctionalInterface` annotation enables compiler checking.
- It supports functional programming in Java.

Example:

```
@FunctionalInterface
interface Greeting {
    void show();
}

class Main {
    public static void main(String[] args) {
        Greeting g = () -> System.out.println("Hello");
        g.show();
    }
}
```

Output: `Hello`

Common built-in interfaces:

Interface	Purpose
`Predicate`	Tests a condition
`Function`	Accepts input and returns output
`Consumer`	Performs an action on input
`Supplier`	Supplies a value

5. Lambda Expression Fundamentals

Introduction

- Lambda expressions represent a block of behavior.
- They use the arrow operator `->`.
- They can accept zero, one, or multiple parameters.
- Parameter types can often be inferred by Java.
- They require a suitable target type, usually a functional interface.

Examples:

```
// No parameter
() -> System.out.println("Hello");

// One parameter
n -> System.out.println(n);

// Two parameters
(a, b) -> a + b;

// Explicit parameter types
(int a, int b) -> a + b;
```

Advantages: Less code, improved readability, and simpler implementation of functional interfaces.

6. Passing Lambda Expressions as Arguments

Introduction

- Java allows lambda expressions to be passed to methods.
- This allows a method to receive behavior as an argument.
- The method can perform different operations using the same code.
- It reduces the need for separate methods for simple operations.
- It is useful in sorting, filtering, and event handling.

Example:

```
interface Operation {
    int calculate(int a, int b);
}

class Main {
    static void perform(int a, int b, Operation op) {
        System.out.println(op.calculate(a, b));
    }

    public static void main(String[] args) {
        perform(10, 5, (a, b) -> a + b);
        perform(10, 5, (a, b) -> a * b);
    }
}
```

Output:

```
15
50
```

7. Block Lambda Expressions

Introduction

- A block lambda contains multiple statements.
- Its body is enclosed in curly braces `{ }`.
- It is useful for complex calculations.
- It can declare local variables and execute multiple operations.
- A `return` statement is used when a value must be returned from the block.

Example:

```
interface Calculation {
    int calculate(int a, int b);
}

class Main {
    public static void main(String[] args) {
        Calculation c = (a, b) -> {
            int sum = a + b;
            return sum * 2;
        };

        System.out.println(c.calculate(10, 5));
    }
}
```

Output: `30`

Expression Lambda vs. Block Lambda

Expression lambda	Block lambda
Usually contains one expression	Can contain multiple statements
Braces are not required	Uses curly braces `{ }`
Suitable for simple operations	Suitable for complex operations
`(a, b) -> a + b`	Uses statements and possibly `return`

Quick Revision

- Singleton Class: Provides one shared instance.
- Reflection API: Examines class information at runtime.
- Lambda Expression: Short implementation of a functional interface.
- Functional Interface: Contains exactly one abstract method.
- Lambda Fundamentals: Use `->` to separate parameters and body.
- Passing Lambda as Argument: Passes behavior to a method.
- Block Lambda: Uses multiple statements inside braces.